

# CDS View 課程練習 8 ( 期中整合 ) : 航線營收分析

## Lecture

### 這一課要教什麼

這是基礎篇的期中整合題，目標很單純：把 **cds01 ~ cds07** 教過的每一個技巧疊進同一組 **CDS View**，做出一個真正有實用價值的「航線營收分析」報表，並且跟一支用傳統 Open SQL 寫的等效報表程式並排比較，親自感受兩種寫法在可讀性、維護性上的差異——不是紙上談兵的比較，是同一份資料、同一組業務規則，兩種寫法都在這個系統上真的跑過、數字對得起來。

### 需求：航線營收分析

給定一家航空公司 ( **p\_carrid** 參數 )，列出這家公司每一條航線 ( **carrid + connid** ) 的：

- 出發城市 / 抵達城市、航空公司全名 ( **cds03 Association** )
- 該航線總共有幾筆訂位、總營收 ( 票價 × 座位數，**cds02 算術運算** )、平均座位使用數 ( **cds07 聚合** )
- 依總營收分類成 **HIGH / MEDIUM / LOW** 三個等級 ( **cds02 CASE WHEN**，透過 **cds05** 分層設計解決同層別名限制 )

### 三層架構設計

#### Layer 1 : **ZI\_CDS08\_ROUTE\_REVENUE** ( **Interface View**，明細層級 )

```
@AbapCatalog.sqlViewName: 'ZICDS08RREV'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS08: Route Revenue Interface View'
define view ZI_CDS08_ROUTE_REVENUE
  with parameters
    p_carrid : s_carr_id
  as select from sflight as Flight
  association [1..1] to spfli as _Schedule
    on _Schedule.carrid = Flight.carrid
    and _Schedule.connid = Flight.connid
  association [1..1] to scarr as _Carrier
    on _Carrier.carrid = Flight.carrid
{
  key Flight.carrid,
  key Flight.connid,
  key Flight.fldate,

  Flight.price,
  Flight.currency,
  Flight.seatsmax,
  Flight.seatsocc,
```

```

    Flight.price * Flight.seatsocc    as EstimatedRevenue,

    _Schedule.cityfrom,
    _Schedule.cityto,
    _Carrier.carrname    as CarrierName
}
where Flight.carrid = $parameters.p_carrid

```

這一層做的事：**cds04** 的 Parameters ( **p\_carrid** )、**cds03** 的雙重 Association ( 一個接航班排程拿城市、一個接航空公司主檔拿全名，且都是直接消費，不像 cds03 範例那樣故意示範「宣告不消費」 )、**cds02** 的算術運算 ( **EstimatedRevenue** )。

⚠ 這一課實測出來的新發現：**Flight.price \* Flight.seatsocc** 這句乘法完全不需要 **CAST** 就能直接編譯——這跟 cds02 學到的「除法需要先 CAST 成 **abap.decfloat34**」不一樣。原本試著比照除法的做法先 **CAST( Flight.price as abap.decfloat34 )** 反而報錯：

```
CAST PRICE of type CURR to type DECFLOAT34 is not possible
```

原因：**price** 是 **CURR** 型別 ( 金額，跟貨幣欄位 **currency** 綁定 )，這個型別不能直接轉型成 **DECFLOAT34**；但拿掉 **CAST**、直接讓 **CURR × INT2** 相乘，編譯器完全能處理 ( 金額乘以整數座位數，語意上也合理，不需要牽涉浮點數 )。結論：**cds02** 教的「除法要先 **CAST**」限制，不能無條件套用到其他運算子——乘法在這系統上不受同樣限制，遇到新的運算組合要重新實測，不能想當然照搬前面學到的 **workaround**。

## Layer 2 : **ZC\_CDS08\_ROUTE\_REVENUE\_STATS** ( **Composite View** · 聚合層級 )

```

@AbapCatalog.sqlViewName: 'ZCCDS08RRST'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS08: Route Revenue Statistics (Aggregated)'
define view ZC_CDS08_ROUTE_REVENUE_STATS
  with parameters
    p_carrid : s_carr_id
  as select from ZI_CDS08_ROUTE_REVENUE( p_carrid: $parameters.p_carrid )
{
  key carrid,
  key connid,

  cityfrom,
  cityto,
  CarrierName,

  count(*)          as FlightCount,
  sum(EstimatedRevenue) as TotalRevenue,
  avg(seatsocc)      as AvgSeatsOccupied
}
group by carrid, connid, cityfrom, cityto, CarrierName

```

這一層做的事：**cds05** 的疊層參數轉傳（冒號語法）、**cds07** 的 **GROUP BY** 聚合——依航線（**carrid+connid**）分組，把 Layer 1 的明細（一列一筆訂位）彙總成「一條航線一列」。

### Layer 3 : **ZR\_CDS08\_ROUTE\_REVENUE\_REPORT** ( **Report View** · 最終呈現層級 )

```
@AbapCatalog.sqlViewName: 'ZRCDS08RTRP'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS08: Route Revenue Report (Final Layer)'
define view ZR_CDS08_ROUTE_REVENUE_REPORT
  with parameters
    p_carrid : s_carr_id
  as select from ZC_CDS08_ROUTE_REVENUE_STATS( p_carrid: $parameters.p_carrid )
{
  key carrid,
  key connid,
  cityfrom,
  cityto,
  CarrierName,
  FlightCount,
  TotalRevenue,
  AvgSeatsOccupied,

  case
    when TotalRevenue >= 2000000 then 'HIGH'
    when TotalRevenue >= 1000000 then 'MEDIUM'
    else                          'LOW'
  end
    as RevenueTier
}
```

這一層正是 **cds05** 教的「分層設計解決 CASE WHEN 限制」的直接應用：**TotalRevenue** 是 Layer 2 聚合算出來的欄位，如果想在 Layer 2 自己的 **CASE WHEN** 裡引用它做分類，會撞上 **cds02** 發現的「不能引用同層別名」限制——這一課再疊一層 Layer 3，讓 **TotalRevenue** 變成 Layer 3 的合法來源欄位，**CASE WHEN** 才能正常引用。

為什麼沒有加 **Access Control** ( **cds06** )：這一課刻意只疊 **cds01~cds05**、**cds07** 的技巧，沒有把 **cds06** 的 **#CHECK** + DCL Role 也加進來——原因是這個案例本身用 **p\_carrid** 參數限定單一航空公司，如果再疊一層字面值式的 Access Control（例如只准看 **LH**），會跟「這個 View 本來就該讓使用者自己選要查哪家航空公司」的設計意圖衝突，示範起來不自然。這不代表 **Access Control** 不能疊進這種多層架構——如果你要練習，可以把 **cds06** 學到的 **#CHECK** 標在 Layer 1 ( **ZI\_CDS08\_ROUTE\_REVENUE** )，機制完全相容，只是這一課的範例選擇不這樣做。

跟等效 Open SQL 報表程式的對比

**ZR\_CDS08\_LEGACY\_REPORT** 用傳統寫法達到完全一樣的效果：撈明細、手動 **LOOP** 累加、逐筆 **SELECT SINGLE** 查城市名稱跟航空公司全名、手動 **IF/ELSEIF** 分類——這是很多實際專案裡「**CDS** 出現之前」報表程式的典型寫法：

```

LOOP AT lt_flight INTO DATA(ls_flight).
  READ TABLE lt_stats WITH KEY carrid = ls_flight-carrid connid = ls_flight-connid
  INTO ls_stats.
  IF sy-subrc <> 0. "沒找到就新增一行
    ...
  ENDIF.
  ls_stats-flight_count = ls_stats-flight_count + 1.
  ls_stats-total_revenue = ls_stats-total_revenue + ( ls_flight-price * ls_flight-
  seatsocc ).
  ...
  MODIFY lt_stats FROM ls_stats TRANSPORTING ... WHERE ...
ENDLOOP.

LOOP AT lt_stats INTO ls_stats.
  SELECT SINGLE cityfrom, cityto FROM spfli WHERE ... INTO (...).
  SELECT SINGLE carrname FROM scarr WHERE ... INTO ....
  IF ls_stats-total_revenue >= 2000000.
    ls_stats-revenue_tier = 'HIGH'.
  ELSEIF ...
  ENDIF.
ENDLOOP.

```

兩種寫法執行結果完全一致（這一課的驗證程式已經逐筆比對過），但可讀性/維護性有幾個具體差異：

|        | 三層 CDS View                                    | 傳統 Open SQL 報表                                                             |
|--------|------------------------------------------------|----------------------------------------------------------------------------|
| 分組邏輯   | <b>GROUP BY</b> 一行宣告完成                         | 要自己寫 <b>READ TABLE</b> 判斷「這個群組存不存在」+ <b>MODIFY ... WHERE</b> 累加，邏輯分散在迴圈裡   |
| 關聯資料查詢 | Association 宣告一次，哪一層要用就直接引用                    | 每一列都要重新 <b>SELECT SINGLE</b> （這一課的例子是 5 條航線就要跑 10 次額外查詢：5 次查城市、5 次查航空公司名稱） |
| 重用性    | Layer 1 / Layer 2 可以被其他報表 / App 直接重用（不用複製貼上邏輯） | 這段邏輯整個寫死在這支程式裡，其他程式想要同樣的彙總資料，只能複製貼上或整個重新設計                                 |
| 可讀性    | 三個 View 各自職責單一，一眼看出「這一層在做什麼」                   | 商業邏輯（分類門檻）、資料存取（ <b>SELECT</b> ）、迴圈控制混在一起，要完整讀完整段程式碼才知道在做什麼                |
| 正確性風險  | <b>GROUP BY</b> 的分組邏輯由資料庫引擎保證正確                | 手動維護「有沒有這個群組」的判斷，容易寫錯（例如忘記在 <b>MODIFY ... WHERE</b> 補全部要累加的欄位）             |

這正是這門課從 cds01 開始就不斷強調的：**CDS View** 的價值不是「查詢邏輯本身變快」，是把查詢邏輯變成一個可以重用、可以疊層設計、交給資料庫引擎保證正確性的 **Repository** 物件——這一課用一個完整、跑得動的案例，把這句話兌現成具體的程式碼對比。

Eclipse ADT 建立順序：Step by Step

1. **ZI\_CDS08\_ROUTE\_REVENUE** ( Layer 1 ) : 以 **SFLIGHT** 為來源，注意 **Flight.price \* Flight.seatsocc** 不要加 **CAST**
2. **ZC\_CDS08\_ROUTE\_REVENUE\_STATS** ( Layer 2 ) : 以 **ZI\_CDS08\_ROUTE\_REVENUE** 為來源，**GROUP BY** 要包含所有非聚合欄位
3. **ZR\_CDS08\_ROUTE\_REVENUE\_REPORT** ( Layer 3 ) : 以 **ZC\_CDS08\_ROUTE\_REVENUE\_STATS** 為來源，**CASE WHEN** 引用 **TotalRevenue**
4. 三層必須依序啟用 ( Layer 1 → Layer 2 → Layer 3 )，任何一層失敗都會導致下一層無法解析
5. 對 **ZR\_CDS08\_ROUTE\_REVENUE\_REPORT** 用 Data Preview ( 會跳出 **p\_carrid** 輸入框，填 **LH** 效果最明顯——這家公司的測試資料涵蓋 HIGH/MEDIUM/LOW 三個等級都有 )

期中整合完成，回顧一下

到這裡，基礎篇的八個主題全部在這個案例裡出現過一次：cds01 ( View 是什麼 )、cds02 ( 欄位運算，含這一課新發現的乘法不需要 **CAST** )、cds03 ( Association )、cds04 ( Parameters )、cds05 ( 分層設計，解決 **CASE WHEN** 限制 )、cds07 ( 聚合 )——只有 cds06 ( Access Control ) 因為前面說明的原因沒有直接疊進來，但機制完全相容、隨時可以加。接下來進階篇 ( cds09 ~ cds16 ) 會教這個系統之前沒碰過的新主題：Extend View、Custom Entity、Analytical Annotation 深入、Virtual Element、Value Help、Hierarchy，最後 cds16 期末整合再把整個 16 題疊成一個完整案例。

## Eclipse ADT Step by Step ( 重點回顧 )

1. Layer 1 **ZI\_CDS08\_ROUTE\_REVENUE** : Parameters + 雙重 Association ( 消費 ) + 算術運算 ( 乘法不需要 **CAST** )
2. Layer 2 **ZC\_CDS08\_ROUTE\_REVENUE\_STATS** : 疊層參數轉傳 + **GROUP BY** 聚合
3. Layer 3 **ZR\_CDS08\_ROUTE\_REVENUE\_REPORT** : 引用上一層聚合欄位的 **CASE WHEN** 分類
4. 三層依序啟用，Data Preview 用 **p\_carrid = 'LH'** 驗證三種等級都出現

## 學習目標

- 能獨立設計一個三層 CDS View 架構，正確安排「哪個技巧該放在哪一層」
- 能講出這一課新發現的具體限制：CDS 算術運算子的行為因運算子而異 ( 除法要求浮點數型別、乘法對 **CURR × INT2** 完全不受這個限制 )，遇到新的型別/運算組合要實測，不能直接照搬其他運算子學到的 **workaround**
- 能寫出一個完整的多層 **GROUP BY** 聚合 + 跨層 **CASE WHEN** 分類的案例
- 能具體列出「三層 CDS View」相對「傳統 Open SQL 手動累加報表」的至少三個可讀性/維護性優勢，並用自己動手驗證過的程式碼佐證，不是憑空背誦
- 知道「為什麼這一課沒有疊 Access Control」的判斷依據 ( 跟 Parameters 設計意圖是否衝突 )，並知道機制上完全相容、可以自行加上

## 物件清單

| 物件                                | 名稱                                   | 型別      |
|-----------------------------------|--------------------------------------|---------|
| CDS Interface View ( Layer 1 )    | <b>ZI_CDS08_ROUTE_REVENUE</b>        | DDLS/DF |
| CDS Composite View ( Layer 2，聚合 ) | <b>ZC_CDS08_ROUTE_REVENUE_STATS</b>  | DDLS/DF |
| CDS Report View ( Layer 3，最終分類 )  | <b>ZR_CDS08_ROUTE_REVENUE_REPORT</b> | DDLS/DF |

| 物件                     | 名稱                     | 型別     |
|------------------------|------------------------|--------|
| 等效 Open SQL 報表 ( 對比用 ) | ZR_CDS08_LEGACY_REPORT | PROG/P |
| 驗證程式                   | ZR_CDS08_DEMO          | PROG/P |

全部物件都在 **\$TMP** 套件，已建立並啟用 ( 讀取 active 版本內容與寫入內容逐字相符 ) 。

## 動手練習

**輪到你了：**這一課沒有另外設計新的動手練習物件——**期中整合題本身就是這一課的動手練習**。建議你依照上面的 Eclipse Step by Step，自己從頭建一次這三層 View ( 不用照抄物件名稱，換一組自己的命名 )，並且：

1. 換一家航空公司 ( 例如 **UA / DL** ) 驗證 Data Preview 結果
2. 試著調整 **RevenueTier** 的門檻數字，觀察分類結果怎麼變化
3. 如果你想挑戰自己，試著把 cds06 學過的 Access Control ( **#CHECK** + DCL Role ) 疊進 Layer 1，看看整條鏈路還能不能正常運作

建好、啟用成功後跟我說一聲，我會幫你核對語法跟設計是否合理。

## 驗證方式

**ZR\_CDS08\_DEMO** 透過 **programrun** 無頭驗證三層 CDS View 查詢結果，並跟 **ZR\_CDS08\_LEGACY\_REPORT** ( 等效 Open SQL 報表，分開執行 ) 的結果逐筆比對：

**三層 CDS Report ( **p\_carriid** = 'LH' )：**

```
LH  0400  FRANKFURT -> NEW YORK   Lufthansa   16  2,151,305.97  201.38  HIGH
LH  0401  NEW YORK   -> FRANKFURT   Lufthansa   15  1,808,856.00  181.07  MEDIUM
LH  0402  FRANKFURT -> NEW YORK   Lufthansa   15  3,428,568.00  343.20  HIGH
LH  2402  FRANKFURT -> BERLIN     Lufthansa   15  1,153,856.00  317.87  MEDIUM
LH  2407  BERLIN    -> FRANKFURT   Lufthansa   15   304,920.00   84.00   LOW
=== Sanity check ===
MATCH: RevenueTier correctly derived through all 3 layers, route count 5
```

**等效 Open SQL 報表 ( **ZR\_CDS08\_LEGACY\_REPORT**，同樣 **p\_carriid** = 'LH' )：**

```
LH  0400  FRANKFURT -> NEW YORK   Lufthansa   16  2,151,305.97  201.38  HIGH
LH  0401  NEW YORK   -> FRANKFURT   Lufthansa   15  1,808,856.00  181.07  MEDIUM
LH  0402  FRANKFURT -> NEW YORK   Lufthansa   15  3,428,568.00  343.20  HIGH
LH  2402  FRANKFURT -> BERLIN     Lufthansa   15  1,153,856.00  317.87  MEDIUM
LH  2407  BERLIN    -> FRANKFURT   Lufthansa   15   304,920.00   84.00   LOW
```

兩份報表逐筆數字完全一致 ( **TotalRevenue** / **FlightCount** / **AvgSeatsOccupied** / **RevenueTier** 全部相符 )，證實三層 CDS View 架構跟傳統手寫報表在業務邏輯正確性上是等價的，差異純粹在程式碼組織方式跟可維護性。

**動手練習的驗證方式：**Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法跟設計。

## 思考題

1. 如果之後想再新增一支「依城市分析營收」的報表（不是依航空公司），這一課的三層架構裡，哪一層可以直接重用、哪一層要重新設計？
2. `ZR_CDS08_LEGACY_REPORT` 對每一條航線都額外跑了一次 `SELECT SINGLE` 查城市、一次查航空公司名稱——如果測試資料從 5 條航線變成 5000 條，這種寫法會有什麼具體的效能疑慮？CDS Association 的做法在同樣情境下會不會有一樣的疑慮？（提示：回顧 cds03「Association 只有被引用才轉譯成 JOIN」）
3. 這一課的 `RevenueTier` 門檻（`2000000 / 1000000`）是寫死在 Layer 3 的。如果不同航空公司應該有不同的分級門檻（例如小型航空公司的「HIGH」標準本來就比大型航空公司低），你會怎麼改這個設計？（提示：回顧 cds04 Parameters）
4. 回顧整個基礎篇（cds01 ~ cds08），你覺得哪一課學到的技巧、或哪一個實測發現的限制，對你來說最意外 / 最有價值？

## 答案

見 `zi_cds08_route_revenue.ddls.abap`、`zc_cds08_route_revenue_stats.ddls.abap`、`zr_cds08_route_revenue_report.ddls.abap`、`zr_cds08_legacy_report.prog.abap`、`zr_cds08_demo.prog.abap`。SAP 端物件：`ZI_CDS08_ROUTE_REVENUE`（Layer 1）、`ZC_CDS08_ROUTE_REVENUE_STATS`（Layer 2）、`ZR_CDS08_ROUTE_REVENUE_REPORT`（Layer 3）、`ZR_CDS08_LEGACY_REPORT`（對比用）、`ZR_CDS08_DEMO`（驗證程式）。這一課沒有另外的動手練習答案快照——期中整合題本身就是練習，鼓勵你自己從頭建一次。